

PharmaRobot ROS 2 Patch Explanation

Detailed explanation of implemented architecture, safety, deployment, and driver robustness changes.

Scope: this document explains the patches and direct edits implemented so far. It does not include Git workflow details or patch validation status. The last section lists remaining validation tests required before marking all implemented changes as fully proven on hardware.

1. Big-picture architecture change

The original robot stack allowed several runtime components to interact directly with the motor driver. This is common during early prototyping because it makes testing easy: a joystick node, a script, or a navigation node can all publish directly to `/cmd_vel`. The problem is that this is not deterministic once more than one publisher exists. ROS 2 topics are many-to-many by design, so the subscriber does not inherently know which publisher is authoritative. For a mobile robot, this creates a safety and control-quality problem: two different sources can alternate commands, overwrite each other, or leave stale commands active.

The new architecture introduces an explicit command path: joystick commands go to `/cmd_vel/joy`, optional test commands go to `/cmd_vel/test`, future navigation commands go to `/cmd_vel/nav`, and a command arbiter chooses exactly one output on `/cmd_vel/safe`. The Roboteq driver subscribes only to `/cmd_vel/safe`. This moves the system from implicit command ownership to explicit command ownership.

```
/cmd_vel/joy priority 100 \  
/cmd_vel/test priority 50 -> command_arbiter -> /cmd_vel/safe -> roboteq_ros2_driver  
/cmd_vel/nav priority 10 /
```

The result is a control chain where every motion command to the motor driver must pass through one known node. This improves traceability, reduces accidental command conflicts, and creates one place where future safety policies can be added.

2. Command arbiter

What was implemented

- New `command_arbiter` package.
- Separate input topics for joystick, test, and navigation sources.
- Priority selection, with joystick highest, test second, navigation lowest.
- Per-source freshness timeout.
- Zero output when no valid source exists.
- Roboteq driver moved to the protected `/cmd_vel/safe` output.

Theory and justification

In control systems, an actuator should normally have one authoritative command stream. Multiple command writers to the same actuator input create a race condition. ROS 2 does not provide exclusive ownership of a topic by default, so a subscriber can receive messages from several publishers without knowing which source is intended to dominate. An arbiter is a software multiplexer: it receives candidate commands, assigns each source a priority and freshness window, and publishes one selected command.

The freshness window is as important as priority. A high-priority message that is old should not keep controlling the robot. Without source timeout, a single old joystick command could remain the selected command even after the joystick disconnects or the node crashes. The arbiter therefore only treats a source as valid if it has published recently.

This improves code quality in three ways. First, it reduces coupling: the motor driver no longer needs to know whether a command came from joystick, test, or navigation. Second, it improves testability: the arbiter can be

tested with synthetic messages without connecting motors. Third, it improves extensibility: Nav2, autonomous tests, or emergency overrides can be added later without changing the motor driver interface again.

3. L1 hold-to-run deadman

What was implemented

- joy_to_cmdvel now requires L1, button index 4, for non-zero joystick motion.
- Stick movement without L1 produces zero velocity.
- Releasing L1 while moving immediately sends a stop command.
- Joystick message loss still triggers timeout stop.
- enable_button_index parameter added so the button mapping can be changed.

Theory and justification

A hold-to-run deadman is a fail-safe human-machine interface pattern. The key idea is that motion requires continuous human intent. A momentary button press is not enough; the operator must keep the enable button held. If the operator releases the button, loses grip, drops the controller, or becomes distracted, the system stops.

This is different from a toggle enable. A toggle can remain enabled after the human stops actively commanding it. A hold-to-run input is safer because the default state is stopped. For a mobile robot under manual testing, this is a practical layer of safety above the physical emergency stop.

In the code, the joystick node already converts axes to a geometry_msgs/Twist. The deadman check is placed before non-zero Twist output. This is the correct layer because it prevents unsafe joystick commands before they enter the arbiter. The arbiter still provides a second layer of freshness timeout downstream.

This improves code quality by separating operator-intent validation from motor-driver logic. The driver does not need to know which joystick button is held; it only receives a valid safe command or a zero command.

4. Joystick-loss and driver command timeouts

What was implemented

- joy_to_cmdvel publishes stop when /joy messages become stale.
- Roboteq driver sends zero command when upstream command messages become stale.
- Driver shutdown now sends explicit zero motor commands before closing the serial port.

Theory and justification

Robots should fail stopped, not fail moving. In a distributed ROS graph, a missing message can mean several things: joystick unplugged, node crashed, network issue, CPU overload, launch process killed, or DDS discovery problem. If a robot only reacts when messages arrive, it can miss the fact that messages have stopped. Timeouts solve this by converting silence into an explicit stop.

There are two timeout layers. The joystick timeout handles loss of /joy before joystick commands are generated. The driver command timeout handles loss of the entire upstream command chain, including arbiter failure. This layered design is valuable because no single node becomes the only stopping mechanism.

The explicit shutdown stop is also important. Closing a serial port is not the same as commanding zero torque or zero speed. Sending a stop command before shutdown reduces the chance that the controller continues executing the last accepted command while the ROS process exits.

5. Reverse steering

What was implemented

- Added reverse_steering parameter.
- When linear velocity is negative, joystick angular steering is inverted.
- Forward steering and pure rotation remain unchanged.

Theory and justification

Differential-drive robots and cars use different operator mental models. In ROS, positive angular.z means a positive yaw direction regardless of whether the robot is moving forward or backward. That mathematical convention is consistent for kinematics, but it can feel wrong to a human using a joystick with car-like expectations. In a car, steering left while reversing makes the vehicle follow a reverse-left path from the driver perspective, even though the instantaneous yaw sign relationship differs from forward motion.

The reverse_steering change adapts the joystick interface, not the low-level robot kinematics. That is the correct design choice. The motor driver should keep using consistent differential-drive math; the joystick node should translate human input into the desired command semantics. Pure rotation is left unchanged because there is no forward or backward travel direction to reinterpret.

This improves usability and reduces operator error. When a robot behaves according to the operator mental model, manual testing becomes safer and more repeatable.

6. Persistent Docker image and dependency fix

What was implemented

- Dockerfile package order fixed.
- ros-humble-joy-linux added as an image dependency.
- Versioned Docker image built and used instead of relying on manual apt-get inside a running container.
- joy_linux availability confirmed after container recreation.

Theory and justification

A container should be treated as disposable. Any package installed manually inside a running container is not part of the deployment definition. If the container is recreated, the manual installation disappears. This is exactly what happened with joy_linux: the current container worked, but restarting the container would remove the dependency and break the control launch.

The solution is to put runtime dependencies into the Dockerfile and build a versioned image. A versioned image creates a reproducible environment: the same tag should contain the same installed packages, system libraries, and ROS dependencies. This makes debugging much easier because the environment becomes part of the controlled artifact, not a hidden manual step.

The Dockerfile order also matters. Ubuntu repository setup, ROS repository setup, and package installation must occur in a valid sequence. Installing ROS packages before the ROS repository is configured produces package-not-found or apt failures. Fixing the order improves repeatability of the build.

7. Systemd and bring-up supervision

What was implemented

- Control and LiDAR stacks split into separate services.
- Foreground launch scripts used instead of detached docker exec processes.
- Stop scripts target only their specific launch/process set.
- Expected SIGINT/SIGTERM shutdown codes handled as successful service exits.

Theory and justification

A service manager can only supervise processes it owns or tracks. The previous pattern of systemd running a short script that started detached docker exec processes caused a mismatch: systemd could report active/exited while the actual ROS nodes were dead, or report failed while nodes were still running. Foreground launch improves observability because systemd tracks the lifecycle of the ROS launch process.

Splitting control and LiDAR into independent services reduces blast radius. Restarting the joystick or Roboteq control stack should not kill the LiDAR stack. Restarting LiDAR should not affect motor control. This separation makes troubleshooting easier and reduces unintended side effects.

Specific stop scripts are safer than broad process killing. A broad pattern like killing all ros2 launch processes can terminate unrelated launches. Specific process patterns reduce collateral damage. This is especially important once SLAM, RViz, logging, or future navigation nodes are added.

8. Stable USB device names

What was implemented

- udev-rule generation tooling for stable serial names.
- Stable names planned: /dev/roboteq, /dev/lidar_front, /dev/lidar_back.
- USB checker script added.
- Runtime configuration prepared to use stable names rather than raw ttyUSB numbers.

Theory and justification

/dev/ttyUSB0, /dev/ttyUSB1, and /dev/ttyUSB2 are enumeration names. They depend on device discovery order. Rebooting, reconnecting a USB cable, or changing hubs can swap these numbers. For a robot, a swapped serial port is dangerous: a motor driver may try to open a LiDAR port, or a LiDAR launch may consume the Roboteq controller port.

udev solves this by creating stable symbolic links based on device identity, such as serial number or physical USB path. When the robot code opens /dev/roboteq, the operating system resolves it to the correct ttyUSB device. This separates application configuration from kernel enumeration order.

Using stable names improves robustness and operational clarity. A technician can inspect /dev/roboteq and know which physical device it represents. The Docker container can also be mapped to semantic device names rather than fragile ttyUSB numbers.

9. Serial protocol robustness

What was implemented

- RoboteqSerialTransport class added.
- Serial reads and writes centralized.
- flushInput removed from the command path.
- std::stoi replaced with strict parser validation.
- Bounded reads, transaction timeouts, response-size limits, and complete-write checks added.
- Acknowledgment and rejection handling added.
- Consecutive failure tracking and reconnection handling added.
- Motor commands are inhibited while the serial link is unhealthy.
- Parser tests added for encoder response handling.

Theory and justification

Serial communication is byte-stream based. It does not inherently preserve message boundaries in the same way a typed ROS topic does. A program must define transactions: what command was sent, what response is

expected, how long to wait, how many bytes are acceptable, and how to parse the result. Without a transaction layer, the code can accidentally mix old bytes with new replies, block too long, or silently discard useful fault information.

flushInput is dangerous in a protocol-driven controller because it can delete information the controller sent, including acknowledgments, errors, or fault messages. The new approach reads bounded responses and interprets them. This preserves observability.

std::stoi throws exceptions on invalid strings and allows loosely formatted input. For a robot driver, parser failure should be explicit and controlled. Strict parsing with range validation makes malformed controller responses a recoverable communication error rather than a crash path.

Acknowledgment handling matters because writing bytes to Linux does not prove the motor controller accepted the command. The operating system can accept the bytes into a buffer even if the controller later rejects the command. Checking acknowledgments and rejections turns the driver from best-effort serial writing into a verified transaction system.

Reconnection logic improves fault tolerance. USB serial links can be disconnected or reset. A robust driver should detect failures, stop accepting motion commands while communication is unhealthy, attempt reconnection, reconfigure the controller, and require a fresh valid command before moving again. This behaviour improves safety and makes faults diagnosable.

10. Optional test and logging packages disabled by default

What was implemented

- Optional logger/test packages kept in the tree but disabled by COLCON_IGNORE.
- odom_test_logs and robot_logs kept available without automatic runtime launch.
- Optional test tooling separated from normal bring-up.

Theory and justification

Logging and calibration tools are useful during development but should not run automatically in the normal robot stack unless explicitly requested. Extra nodes can consume CPU, write to disk, publish topics, or interfere with command routing. Keeping them in the tree but disabled by default preserves developer utility without adding runtime risk.

This also supports cleaner builds. The core robot stack can be built and launched quickly, while optional test packages can be enabled only for calibration sessions.

11. Remaining validation tests

The following tests are still required to fully validate the combined patch set on the real robot. Some software tests may already have been run during development, but the list below is the full validation checklist to repeat after all patches are combined and after container recreation.

Remaining Validation Tests for Implemented Patches

1. Baseline ROS graph validation after a clean start

Command:

```
ros2 node list | sort
```

```
ros2 node list | sort | uniq -d
```

Expected:

- Control nodes present: /joy_node, /joy_to_cmdvel, /command_arbiter, /roboteq_ros2_driver.
- LiDAR nodes present when LiDAR service is active: /rplidar_front, /rplidar_back, /base_to_front_laser, /base_to_back_laser.
- Duplicate node list is empty.

2. Command arbiter software integration test

Command:

```
/ros_ws/deployment/scripts/test_command_arbiter.sh
```

Expected:

- Navigation source selected when alone.
- Test source overrides navigation.
- Joystick source overrides test and navigation.
- Stale joystick releases to test.
- Stale test releases to navigation.
- All stale sources produce a stop command.

3. L1 deadman software integration test

Command:

```
/ros_ws/deployment/scripts/test_joystick_deadman.sh
```

Expected:

- Stick motion is blocked while L1 is released.
- L1 enables a non-zero joystick command.
- Releasing L1 sends an immediate stop.
- Joystick message loss sends a timeout stop.

4. Physical L1 deadman validation

Setup:

- Robot wheels lifted or robot secured in a clear test area.
- Monitor `/cmd_vel/safe`.

Expected:

- Moving the stick without L1 keeps `/cmd_vel/safe` at zero.
- Holding L1 and moving the stick produces non-zero `/cmd_vel/safe`.
- Releasing L1 while moving immediately returns `/cmd_vel/safe` to zero.
- Disconnecting the joystick while enabled returns `/cmd_vel/safe` to zero after the timeout.

5. Reverse steering physical validation

Setup:

- Robot wheels lifted first, then controlled floor test.

Expected:

- Forward + left turns left.
- Forward + right turns right.
- Reverse + left behaves like car-style reverse-left steering.
- Reverse + right behaves like car-style reverse-right steering.
- Pure rotation left/right remains unchanged.

6. Persistent Docker image recreation test

Command sequence:

- Restart/recreate `pharmarobot.service`.
- Do not run `apt-get` manually.
- Verify `joy_linux` and `command_arbiter` are still available.

Expected:

- ros2 pkg prefix joy_linux returns /opt/ros/humble.
- ros2 pkg executables joy_linux lists joy_linux_node.
- ros2 pkg prefix command_arbiter returns /ros_ws/install/command_arbiter.
- Control service starts without package-not-found errors.

7. Control service supervision test

Command:

```
systemctl restart pharma-minimal-nodes.service
```

```
systemctl status pharma-minimal-nodes.service --no-pager -l
```

Expected:

- Service becomes active/running.
- ROS nodes remain children of the supervised launch process.
- Normal stop/restart does not leave a persistent failed state.

8. LiDAR/control independence test

Command sequence:

- Start both control and LiDAR services.
- Restart control service only.
- Verify LiDAR nodes remain alive.
- Restart LiDAR service only.
- Verify control nodes remain alive.

Expected:

- Restarting one stack does not kill the other stack.

9. Stable USB name generation test

Command:

```
install_usb_udev_rules.sh with the current Roboteq/front/back mapping.
```

```
check_usb_devices.sh
```

```
readlink -f /dev/roboteq /dev/lidar_front /dev/lidar_back
```

Expected:

- /dev/roboteq resolves to the Roboteq controller.
- /dev/lidar_front resolves to the front LiDAR.
- /dev/lidar_back resolves to the back LiDAR.

10. Stable USB reboot test

Command sequence:

- Reboot the NUC.
- Run check_usb_devices.sh.
- Start robot services.
- Check Roboteq and LiDAR parameters.

Expected:

- Stable symlink names still point to the correct devices.
- Roboteq node uses /dev/roboteq.
- Front LiDAR uses /dev/lidar_front.
- Back LiDAR uses /dev/lidar_back.

11. Stable USB unplug/replug test

Setup:

- Stop control and LiDAR services before unplugging serial devices.
- Replug devices into the same physical USB ports.
- Recreate the container if Docker device mappings need to refresh.

Expected:

- Stable symlink names still resolve correctly.
- Robot services use the stable names rather than changed ttyUSB numbers.

12. Roboteq parser-only unit test

Command:

```
colcon test --packages-select roboteq_ros2_driver --ctest-args -R test_roboteq_protocol
```

Expected:

- Parser test passes with 0 errors and 0 failures.
- Full package lint may still report style issues until a separate style cleanup is performed.

13. Roboteq startup acknowledgment test

Command:

```
journalctl -u pharma-minimal-nodes.service --no-pager -n 120
```

Expected:

- Roboteq serial connection is open, configured and ready.
- Motor-controller configuration commands were acknowledged.
- No repeated acknowledgment timeout, command rejection, or response deadline messages.

14. Encoder stability test

Command:

```
ros2 topic echo /wheel_ticks
```

```
ros2 topic hz /wheel_ticks
```

Expected:

- /wheel_ticks continues publishing.
- Frequency remains stable.
- No serial parser exceptions appear in the service log.

15. Wheel-lifted movement test after serial-layer change

Setup:

- Wheels lifted.
- L1 held only during intentional motion.

Expected:

- Forward, reverse, rotation, and steering commands move the wheels correctly.
- Releasing L1 stops the wheels immediately.
- No serial failures are logged during normal operation.

16. Roboteq USB disconnect/reconnect recovery test

Setup:

- Wheels lifted.
- L1 released.
- Monitor pharma-minimal-nodes.service logs.

Expected after disconnect:

- Driver logs serial failure without crashing.
- Driver stops accepting motor commands while serial is unhealthy.
- Reconnection attempts occur periodically.

Expected after reconnect:

- Driver reopens the serial port.
- Controller configuration is reapplied and acknowledged.
- Robot remains stopped until a new valid L1-enabled command is sent.

17. End-to-end regression test

Command sequence:

- Recreate container from persistent image.
- Start control and LiDAR services.
- Run command arbiter and deadman software tests.
- Verify ROS graph, RViz LiDAR fixed frame, joystick motion, wheel ticks, and service status.

Expected:

- All previously working behaviours still work after all patches are combined.